

Project Proposal

Multi-Process Video Encoding System Operating Systems

Students Name	Faraz Ahmed, Saad Aamer, Muhammed Talha Aamir
Roll Number	24k0942, 23k0921, 24k0729
Course	Operating Systems
Institution	FAST-NUCES
Submission Date	1 May 2026

1. Executive Summary

This proposal outlines the design and implementation of a Multi-Process Video Encoding System, a simulated video encoding pipeline that demonstrates core Operating Systems concepts including process management, thread-based parallelism, mutual exclusion, and semaphore-based synchronization.

The system orchestrates multiple concurrent worker processes, each spawning internal threads to encode individual video frames. A mutex-protected shared output buffer aggregates results safely, while POSIX semaphores coordinate pipeline stage completion. The final product includes a real-time encoding monitor GUI displaying system-wide progress and performance metrics.

2. Problem Statement

Modern video encoding is computationally intensive and inherently parallel. Frames can be encoded independently, pipeline stages can run concurrently, and multiple encoding profiles can execute simultaneously. Implementing this in a systems programming context requires careful coordination of:

- Concurrent processes that must not corrupt each other's state
- Threads within each process competing for a shared output buffer
- Ordered stage completion across process boundaries
- Fault tolerance when individual worker processes fail

This project addresses these challenges using POSIX-standard OS primitives, providing practical experience with synchronization mechanisms covered in course theory.

3. Objectives

3.1 Mandatory Objectives

- Implement a multi-process pipeline using `fork()` with a parent orchestrator managing N worker processes via `waitpid()`
- Spawn `pthread`s within each worker process for frame-level parallel encoding
- Protect the shared output buffer using `pthread_mutex_t` to prevent data races
- Use POSIX named semaphores to coordinate completion between pipeline stages (Read -> Encode -> Write)

3.2 Additional Objectives

- Support multiple encoding profiles (High Quality, Fast, Balanced) with configurable compression parameters
- Implement a load balancer that distributes frame batches to worker processes based on utilization
- Collect and display performance metrics: throughput (frames/sec), per-stage latency, CPU utilization
- Implement error recovery to detect and respawn failed worker processes without halting the pipeline
- Produce a structured log file with timestamped events for debugging and audit purposes

4. System Architecture

4.1 Overview

The system follows a three-stage producer-consumer pipeline. Each stage runs in a separate process. The orchestrator (main process) manages the lifecycle of all worker processes and monitors pipeline health.

```
Main Process (Orchestrator)
    | fork() x N
Worker 1      Worker 2      ...      Worker N
    | pthread x T per process
Frame Threads -> mutex -> Output Buffer -> sem_post()
```

4.2 Component Roles

- Main Process: Spawns N worker processes via fork(). Monitors health via waitpid() with WNOHANG. Respawns failed workers. Aggregates final metrics.
- Worker Process: Spawns T frame-encoding threads using pthreads. Each thread independently encodes one assigned frame.
- Output Buffer: Accumulates encoded frame data from all threads within a process. All writes are serialized via pthread_mutex_lock/unlock.
- Stage Semaphores: Named semaphores (sem_open) shared across processes. Stage N calls sem_post() to unblock Stage N+1.

5. Technology Stack

Component	Technology	Purpose
Process Management	fork(), waitpid(), exit()	Worker process spawning and lifecycle
Thread Management	pthreads (pthread_create, pthread_join)	Frame-level parallel encoding
Mutual Exclusion	pthread_mutex_t	Shared output buffer protection
Stage Sync	POSIX semaphores (sem_open/post/wait)	Pipeline stage coordination

IPC	Shared memory (shmget / mmap)	Cross-process state and metrics
GUI	Python + tkinter / ncurses	Live encoding monitor dashboard
Build System	GNU Make	Compilation and linking
Language	C (C11 standard)	Core implementation

6. Synchronization Design

6.1 Mutex Usage

Each worker process maintains a single `pthread_mutex_t` guarding its output buffer. All encoding threads call `pthread_mutex_lock()` before appending a completed frame result, and `pthread_mutex_unlock()` immediately after. The critical section is kept minimal to maximize thread throughput.

6.2 Semaphore Usage

Three named POSIX semaphores are created by the orchestrator: `sem_stage1_done`, `sem_stage2_done`, and `sem_pipeline_complete`. When a stage process finishes its assigned batch, it calls `sem_post()`. The subsequent stage blocks on `sem_wait()` until signaled. This ensures strict ordering without busy-waiting.

6.3 Error Recovery

The orchestrator polls worker status every 500ms using `waitpid(pid, &status, WNOHANG)`. If a worker exits unexpectedly with a non-zero status, the orchestrator logs the failure, re-queues the affected frame batch, and forks a replacement worker. Semaphore state is reset to prevent deadlock.

7. GUI — Encoding Monitor

The GUI is implemented as a Python script that reads live metrics from a shared memory segment updated by the C backend. It displays:

- Per-process progress bars showing frames encoded out of total frames
- Pipeline stage indicators (Idle / Active / Complete)
- Real-time throughput graph (frames per second over time)
- Worker process health status with PID display
- Global encoding completion percentage and estimated time remaining

The GUI refreshes every 250ms using Python tkinter, with a fallback ncurses terminal monitor for headless environments.

9. Expected Outcomes

Upon completion, the project will deliver:

- A working C codebase demonstrating multi-process and multi-thread video encoding simulation
- Correct mutex and semaphore synchronization with no data races or deadlocks
- A live GUI encoding monitor showing real-time pipeline state
- A structured log file and performance report comparing single-process vs multi-process throughput
- Complete source code with Makefile, README, and inline documentation

